AI-Powered Resume Builder

Project Overview :

This project is an AI-based resume builder that helps users create professional resumes easily. It takes user details and a job description as input and generates a clean, job-specific resume. The system uses Generative AI to convert raw information into a structured and ATS-friendly resume.

Project Statement :

Many students and freshers find it difficult to create a proper resume.
They are often unsure about what to include, how to structure it, and how to match it with specific job roles.

As a result, they create generic resumes that do not align with job requirements.
These resumes miss important skills and keywords, leading to rejection in ATS systems and reducing their chances of getting selected.

Therefore, there is a need for a system that can help users create professional, job-specific resumes and guide them in improving their skills.

Solution :

This system uses AI to generate professional resumes based on user input and job descriptions.
It analyzes the user's profile and creates a resume that matches the job requirements.
It also helps users understand what skills are needed for a specific role.

Objectives :

The main goal of this project is to help students create better resumes using AI.
It focuses on generating professional and job-specific resumes quickly and easily.

The objectives of this project are:

- To help users create a proper and professional resume
- To generate resumes based on job requirements
- To improve the chances of getting selected
- To save time by automating resume creation
- To build a real-world AI-based application

Technologies Used :

This project is built using the following technologies:

- Streamlit – Used to create the frontend interface where users enter their details
- FastAPI – Used to build the backend API for handling requests and processing data
- Groq (LLM) – Used as the AI model to generate resume content
- LangChain – Used to connect and manage interactions with the AI model
- ReportLab – Used to generate resumes in PDF format
- Python – Main programming language used to develop the entire project

Why These Technologies :

We used these technologies because they make the system easy to build, fast, and efficient.

- Python – It is simple and widely used for AI and backend development
- Streamlit – It helps to quickly create a user interface without complex frontend coding
- FastAPI – It is fast and useful for building APIs to connect frontend and backend
- Groq (LLM) – It provides fast AI models to generate resume content
- LangChain – It helps in managing and interacting with the AI model easily
- ReportLab – It is used to create professional PDF resumes

System Architecture :

The system follows a simple flow where the user gives input, the backend processes it using AI, and the final resume is generated as a PDF.

User

↓

Streamlit (app.py)

↓

HTTP POST Request

↓

FastAPI (main.py)

↓

run_agent()

(LangChain + Groq LLM)

↓

Resume Text Generated

↓

create_pdf()

(ReportLab)

↓

PDF File Created

↓

FastAPI Response

↓

Streamlit (Display + Download Button)

↓

User

1. User fills the form in Streamlit with personal details and job description
2. Streamlit sends the data to FastAPI using an HTTP request
3. FastAPI receives the data and processes it
4. The system sends the data to the AI model (Groq using LangChain)
5. The AI generates the resume content
6. The system converts the content into a PDF using ReportLab
7. FastAPI sends the result back to Streamlit
8. Streamlit displays the resume and provides a download button

Design Workflow :

The system follows a simple workflow to generate resumes using AI.

1. User enters details like education, skills, and experience
2. User provides a job description
3. The system processes the input data
4. AI generates a structured resume
5. The resume is formatted properly
6. A PDF file is created
7. The user can view and download the resume

Project File Structure :

```
Resume Builder/
├── app.py          # Streamlit frontend UI
├── main.py         # FastAPI backend server
├── agent.py        # LLM resume generator + PDF creator
├── .env            # API keys (GROQ_API_KEY)
├── req.txt         # Python dependencies
├── resume.pdf      # Generated resume output
└── doc/            # Python virtual environment
    └── pyvenv.cfg
```

File Explanation (Simple)

- app.py
  Handles the user interface.
  Takes input from user and shows output with download option.
- main.py
  Works as backend API.
  Receives data, processes it, and sends response.
- agent.py
  Core logic of the project.
  Generates resume using AI and creates PDF.
- .env
  Stores API key securely (GROQ_API_KEY).
- req.txt
  Contains all required libraries for the project.
- resume.pdf
  Final generated resume file.

Code Documentation :

agent.py (AI + PDF Core)

This file contains the main logic of the project.

```python
# Load API
from langchain_groq import ChatGroq
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv("GROQ_API_KEY")

llm = ChatGroq(
    model="llama-3.1-8b-instant",
    temperature=0,
    api_key=api_key
)
```

- It connects to the AI model (Groq)
- It creates the prompt using user data
- It generates resume content using AI
- It converts the generated content into a PDF using ReportLab

run_agent() Function

- Takes user input (name, skills, projects, etc.)
- Creates a structured prompt
- Sends the prompt to the AI model
- Receives generated resume text

Prompt Technique

- Uses structured prompts
- Gives clear instructions to AI
- Ensures proper format (headings, bullets)
- Makes output ATS-friendly

create_pdf() Function

- Takes resume text
- Formats it properly
- Converts it into a PDF file

main.py (Backend)

This file handles the backend API.

```python
from fastapi import FastAPI
from pydantic import BaseModel
from agent import run_agent, create_pdf

app = FastAPI()

class UserData(BaseModel):
    name: str
    education: str
    skills: str
    projects: str
    experience: str
    certifications: str
    links: str
    job_description: str

@app.post("/resume")
def generate(data: UserData):

    try:
        result = run_agent(data.dict())

        pdf_file = create_pdf(result["resume"])

        return {
            "resume": result["resume"],
            "pdf_file": pdf_file
        }

    except Exception as e:
        return {"error": str(e)}
```

- Receives user data from frontend
- Calls AI function (run_agent)
- Calls PDF function (create_pdf)
- Sends response back to frontend

app.py (Frontend)

This file handles the user interface.

- Takes input from user
- Sends data to backend
- Displays generated resume
- Provides download button

API Reference

The system uses an API to send user data from the frontend to the backend and get the generated resume.

Endpoint

POST /resume

This API is used to generate the resume.

How It Works

- Streamlit sends user data to the backend
- Backend processes the data
- AI generates the resume
- The system returns resume text + PDF file

Request Body Schema

```
{
  "name":            "string  — Full name of the candidate",
  "education":       "string  — Degree, institution, graduation year",
  "skills":          "string  — Technical skills, tools, frameworks",
  "projects":        "string  — Project titles and descriptions",
  "experience":      "string  — Work experience or internships",
  "certifications": "string  — Certification names and issuers",
  "links":           "string  — LinkedIn, GitHub, portfolio URLs",
  "job_description":"string  — Target job description text"
}
```

Input Data (What user gives)

The API receives:

- Name
- Education
- Skills
- Projects
- Experience
- Certifications
- Links
- Job Description

Output (What system returns)

- Generated resume text
- PDF file of the resume

```
{
  "resume":   "string — Full ATS-formatted resume text",
  "pdf_file": "string — File path to generated resume.pdf"
}
```

Error Handling

If something goes wrong, the system returns an error message.

Error Response

```
{
  "error": "string — Error description if generation fails"
}
```

Resume Output Structure

Kalyan Chitiken

kalyanchitiken@gmail.com | 8688840890 | Hyderabad, India |
https://www.linkedin.com/in/kalyanchitiken | https://github.com/kalyanchitiken

CAREER OBJECTIVE:

To secure a challenging Data Scientist position that utilizes my expertise in Python, Machine Learning, Deep Learning, and NLP to drive business growth and innovation.

PROFESSIONAL TRAINING

Data Science & Generative AI Intern

Innomatics Research Labs | Jan 2025 – Present | Hyderabad

- Worked on Generative AI and LLM-based applications including RAG systems and chatbot development.
- Built agentic workflows using LangGraph for task automation.
- Developed machine learning and deep learning models for real-world problems.
- Deployed AI applications using Streamlit and Hugging Face.

EDUCATION

B.Tech in Computer Science
JNTU Hyderabad

TECHNICAL SKILLS

Programming Languages: Python
Machine Learning: TensorFlow, Scikit-learn, Pandas, NumPy
Deep Learning: LangChain, LangGraph, RAG, Hugging Face
Natural Language Processing: NLP
Database Management: SQL
Deployment Tools: Streamlit, FAISS

PROJECTS

JobMatch AI

- Built an agentic AI system for automated resume screening using LangGraph and LLMs.
- Developed a robust and efficient workflow for resume screening.

Conversational PDF RAG Chatbot

- Developed a document-based chatbot using FAISS and Hugging Face models.
- Implemented a conversational interface for user interaction.

Rainfall Prediction

- Built a classification model using Random Forest to predict rainfall.
- Analyzed and visualized the results to identify trends and patterns.

YouTube Comment Sentiment Analysis

- Developed an LSTM-based deep learning model for sentiment classification.
- Trained and tested the model on a large dataset of YouTube comments.

CERTIFICATES

- Data Science Certification
- Data Analysis Certification
- Generative AI Internship – Innomatics Research Labs

LANGUAGES

English, Telugu, Hindi

- The resume always follows the same structure
- Each section is clearly separated
- Bullet points are used for clarity
- Content is aligned with job description

Setup & Installation :

Prerequisites :

```
fastapi
uvicorn
streamlit
langchain
langchain-gro
python-dotenv
reportlab
requests
pydantic
```

Installation Steps :

Follow these steps to run the project:

1. Download or clone the project
2. Open the project folder
3. Create a virtual environment
4. Activate the environment
5. Install required libraries
6. Add your Groq API key in .env file
7. Start the backend server (FastAPI)
8. Start the frontend (Streamlit)
9. Open the app in browser

Simple Command :

- Install dependencies → pip install -r req.txt
- Run backend → uvicorn main:app --reload

- Run frontend → streamlit run app.py

Dependencies :

Dependencies are the libraries required to run the project.

Important Libraries Used

- FastAPI → backend API
- Uvicorn → runs backend server
- Streamlit → frontend UI
- LangChain → connects AI model
- langchain-groq → Groq API integration
- ReportLab → PDF generation
- Requests → API communication
- Pydantic → data validation
- python-dotenv → loads API key

Challenges Faced & Solutions :

1. AI Output Formatting Issue

Challenge:
The AI was generating messy output with unwanted symbols (like **, random spacing, poor structure).

Solution:
Improved the prompt design and added a cleaning process to remove unwanted symbols and format the text properly.

2. Resume Not Matching Required Structure

Challenge:
The generated resume was not following a proper professional format.

Solution:
Created a strict prompt with clear instructions (sections, bullet points, formatting rules) to guide the AI.

3. API Key Errors & Configuration Issues

Challenge:
The application failed when the API key was missing or not properly configured.

Solution:
Used .env file and Streamlit secrets to securely store and load the API key.

4. PDF Formatting Issues

Challenge:
The generated PDF looked unstructured and not visually appealing.

Solution:
Implemented custom styling logic using ReportLab to format headings, spacing, and bullet points properly.

5. Deployment Issues

Challenge:
The app was not working correctly after deploying on Streamlit Cloud.

Solution:
Fixed dependency issues, added required packages, and configured secrets properly.

6. Backend & Frontend Communication

Challenge:
Errors occurred while sending data between Streamlit and FastAPI.

Solution:
Used proper API structure (POST request) and handled request/response carefully.

Future Scope :

This project can be improved further to make it more useful, smart, and user-friendly.

Short-Term Improvements

- Improve the UI design to make the application more attractive and user-friendly
- Add different resume templates (modern, simple, creative)
- Build a resume scoring system to compare resume with job description
- Add a cover letter generator using the same AI

- Improve input validation to avoid missing fields

Long-Term Improvements

- Connect with LinkedIn to automatically fill user details
- Deploy on cloud (AWS/GCP) with user accounts and resume history
- Create a dashboard to track resume performance
- Support multiple languages
- Add skill gap analysis using AI
- Build an interview preparation assistant

Key Learnings :

Through this project, I learned how to build a complete AI-based application from start to deployment.

What I Learned

- Learned how to work with **Generative AI and LLMs** to generate real-world content
- Understood the importance of **prompt engineering** to get correct and structured output
- Learned how to connect **frontend and backend using APIs**
- Gained experience in building applications using **Streamlit and FastAPI**
- Learned how to generate and format **PDFs using ReportLab**
- Understood how to manage **API keys securely using .env and secrets**
- Learned how to handle **errors and debug issues during development and deployment**
- Gained knowledge about **deployment on cloud (Streamlit Cloud)**

Conclusion :

This project demonstrates how Generative AI can be used to build a real-world application for resume creation.
It helps users generate structured, professional, and ATS-friendly resumes easily.

The system integrates frontend, backend, and AI models to automate the resume creation process and improve user efficiency.